

Neural Macro Compilation: Compiling Slow Construction Paths into Fast Reasoning Paths

> **Draft v0.1 (English)** | 2026-08-11
> Target: ICML/COLM (method paper) | Patent: M1 (semantic-path compilation)
> Evidence: exp50/51/80/90 via run_exp official judge (docs/paper_data/)

Abstract

A model that has learned base operations can often **already execute** a new composite operator by unfolding its definition chain – but along a long construction path (many steps/tokens). We propose **neural macro compilation**: compile the slow construction path into a fast reasoning path within the **same model**, using few automatically generated samples, while (i) preserving semantics exactly (verifiable with fallback) and (ii) never expanding the semantic closure. On a 2-layer, <1 MB transformer: a 2000-digit carry chain executes identically on construction (0.62 s), formal (0.069 s), and intuition (0.086 s) channels (1.000); the compiled fast path generalizes with zero decay to 2000 digits, radix 60, and 100% digit anonymization; step-skipping is visible as 82→5 tokens for a 20-digit addition. Unlike distillation, teacher and student are the same model's slow and fast paths (construction-channel self-compilation); unlike program synthesis, no program is compiled out-of-band.

1 Introduction

****The problem.**** When a model has learned base operations, a new operator σ is already executable: its definition chain unfolds into base operations the model can follow. But the path is long – per-digit iteration, long token sequences – so inference cost scales with construction length, not semantic size.

****Existing remedies are invasive.**** Distillation needs a separate teacher and new parameters. Few-shot fine-tuning risks disturbing learned abilities and gives no equivalence guarantee. Program-to-neural compilation (ICML 2024) treats the program as external input and compiles out-of-band. None exploits that the model **already** has the semantics.

****This work.**** The same model already contains two paths for σ : the slow **construction** path (definition-driven, semantically complete) and a fast path that training can establish in weights. We compile the former into the latter – **neural macro compilation** – with minimal supervision generated automatically by the slow path itself, a verifier, and a runtime fallback. The macro adds speed, not semantics. Relation to P1: P1 establishes **that** the representation supports systematic generalization and **why** (formalized induction); this paper establishes **how** an already-executable operator is re-compiled into a cheap path – the two are complementary, and P2 does not re-derive P1's generalization results.

****Contributions.****

- ****C1 – Semantic-path self-compilation.**** Compile the model's own slow construction path into a fast reasoning path with few self-generated samples, preserving semantics and the semantic closure.
- ****C2 – Zero-sample semantic closure.**** New operators are executable with zero training (construction path); training only reduces cost.
- ****C3 – Verifiable equivalence + fallback.**** Fast outputs checked against slow outputs; failures route back to the slow path.
- ****C4 – Empirical profile.**** 2000-digit carry chain correct on all channels; fast path 7× cheaper at width 2000; zero-decay generalization; 82→5 token step-skipping.

2 Setting

This paper builds directly on the token-native representation of P1: concepts carry structured definitions; a **definition chain** of σ is the transitive closure of concepts σ 's definition references (P1 §2). The compilation targets are channels over this representation.

****Slow path (construction channel).**** Definition-driven unfolding: σ 's definition chain expands into base operations executed step by step (per-digit iteration, carry propagation). Semantically complete – the semantics is guaranteed by definition; cost scales with construction length.

****Fast path (intuition channel).**** Direct execution compiled into weights: input → output in a short sequence, skipping intermediate construction steps. Cost scales with output length, not construction depth.

****Formal path (rewriting channel).**** The definition-driven rewriting engine – shares its definition source with construction; serves as an independent cross-check in verification.

****Goal.**** For a new operator σ with expansion path T_σ :

- $\text{Sem_fast}(\sigma) = \text{Sem_expanded}(T_\sigma)$ – exact semantic equivalence, verifiable;
- $\text{Cost_fast} \ll \text{Cost_expanded}$ – steps/tokens/latency reduced;
- $\text{Closure_fast}(\sigma) = \text{Closure_expanded}(\sigma)$ – no semantic-closure expansion.

****Terminology.**** The fast path is the **reasoning fast path** – intuition: hard to obtain (compilation requires sufficient training; config-dependent epochs), but easy to generalize by skipping steps. The full account is in P1 (§3, §7); here we focus on the compilation method.

3 Method: Neural Macro Compilation

Five stages.

****S1 – Path recognition.**** Expand σ 's definition chain into base operations ($E(\sigma) = T_\sigma$). If σ is expressible via the model's base operations, it is compilable – decided by the definition graph, not a separate synthesizer.

****S2 – Sample generation.**** Use the slow path (construction channel / reference evaluator) to generate (σ, x_i, y_i) pairs automatically. Self-supervised labels: the samples are the model's own semantics. Few samples suffice (tens) because the model already knows the semantics; the samples teach only the **shape** of the direct path. The dependence

of compilation quality on sample count is a falsifiable claim, tested in §5 (currently pending).

****S3 – Parameter update.**** Update a subset of parameters (embedding / adapter / partial / full – graded narrow to wide) so σ maps directly to its result in one short sequence, preserving other abilities (restricted range, or verification after full update).

****S4 – Semantic verification.**** Verify fast-path outputs against slow-path outputs on a held-out set by full output-sequence comparison (the only trusted metric). Only a passing macro is installed.

****S5 – Runtime routing.**** Route σ to the fast path when its execution-cost threshold is met; fall back to the slow path on verification failure. The fast path is an optimization, not a replacement.

****Key distinction from distillation.**** In distillation, a (possibly external) teacher transfers knowledge to a student. Here teacher and student are the **same model's** slow and fast paths: the construction channel supervises its own intuition channel. No new semantic machinery; the macro is compiled from existing competence.

4 Experiments

All via the official run_exp judge path (full-sequence reconstruction). Model: exp02_supervised_s2 (2-layer, dim-64, <1 MB, judgment 1.000 on the full suite). Positioning: P1 establishes that this model generalizes (width/radix/anonymization); this paper establishes that its **already-generalizing** capability can be re-compiled into a cheaper path – the experiments below measure executability, cost, and step-skipping, not generalization itself (which P1 covers).

4.1 Zero-sample semantic closure (C2)

Input	Construction	Formal	Intuition (judgment)
12+34	✓ ✓ ✓		
123456789+987654321	✓ ✓ ✓		
999...9 (20 digits)+1	✓ ✓ ✓		
999...9 (2000 digits)+1	✓ ✓ ✓		**1.000**

Construction and formal channels agree exactly on all four inputs (full output sequences); intuition (the model) scores judgment accuracy 1.000 on the same inputs, including the 2000-digit carry chain – with zero training for that specific operator. Semantics precede training: the definition chain alone makes σ executable.

4.2 Cost profile (C1, C4)

Width	Construction (slow)	Formal	Intuition (fast)	Speedup vs construction
2	0.0005 s	0.0012 s	0.0018 s	–
9	0.0031 s	0.0007 s	0.0043 s	–
2000	**0.6206 s**	**0.0695 s**	**0.0857 s**	**7× (intuition) / 9× (formal)**

At 2000 digits, the fast path is 7× cheaper than construction. Construction grows linearly with digits; the fast path is a short single forward pass. Honest caveat: the formal engine is faster still; the intuition channel's value is end-to-end execution in weights (no external oracle) plus preserved generalization.

4.3 Step-skipping generalization (C4)

At fixed acquisition (8 epochs), the compiled fast path generalizes by skipping construction steps with ****zero decay**** (exp80): 2000-digit width (10× training width), radix 60 (6×), 100% digit anonymization, and joint width × radix – all at 1.000. Acquisition and step-skipping generalization are decoupled: the fast path is hard to obtain but easy to extrapolate.

4.4 Machine form of step-skipping (C4)

Path	Tokens	Time
Construction (digit-wise unfolding)	82	0.410 ms
Intuition (atomic num token)	5	0.186 ms

20-digit addition: the fast path treats the whole numeral as an atomic token (5 tokens) versus 82-token digit-wise unfolding – a 16× token reduction, 2.2× faster. ****Honest caveat****: the atomic token is outside the model vocab (pad id 0); the measurement is the sequence-length effect on forward cost – a lower bound, not a claim of learned atomic-token semantics. If the model truly learned the atomic token, the fast path would be an O(1) decision with a larger advantage.

5 Ablations

Ablation	Question	Status
Sample count (10/20/50)	how few instances suffice for compilation	pending
Update scope (embeddings / adapter / partial / full)	cost-degradation trade-off; original-capability protection	pending
No verifier / no fallback	error propagation without the safety net	pending
Closure invariance (EXP-52)	fast-path executable set = slow-path executable set	pending

The three-channel consistency experiment (exp50) already provides the core verification evidence: construction = formal exactly (2/9/20/2000 digits) and intuition 1.000, including the 2000-digit carry chain. Closure invariance (EXP-52) tests whether the fast path executes exactly the inputs the slow path could execute – the semantic-closure guarantee of Goal (iii).

6 Related Work

- **Program-to-neural compilation** (Learning to Compile Programs to Neural Networks, ICML 2024): builds a neural surrogate for an *external* program from behavioral data; requires a program specification and large data. Difference: here the "program" is the model's own construction path; the teacher is the model itself; no program analysis. Related survey perspective in P1 §9.
- **Knowledge distillation** (Hinton et al., 2015): teacher-student, typically a separate/larger teacher. Difference: same model, two channels; no external teacher; equivalence verified with fallback.
- **Inference compilation / shortcut distillation** (diffusion, 2025): few-sample shortcut distillation compresses long correct paths. Similar surface; difference: semantic equivalence with a verifier + fallback, no semantic-closure expansion.
- **Few-shot fine-tuning**: general; this work is the special case where the capability already exists in the slow path, so few instances *compile* rather than *learn*.
- **Continuous / continual learning**: adding a capability without forgetting old ones. This work's update-scope restriction (S3) and verification (S4) are a forgetting-mitigation mechanism; the ablation (§5) measures the trade-off.
- **Mechanistic / hand-compiled transformers** (e.g. a hand-compiled WASM interpreter): hand-assigned weights interpret a language. Ours is *learned* compilation – gradient training produces the macro, no hand-set weights.
- **Continual learning / catastrophic forgetting**: fine-tuning risks forgetting; our restricted-update + verification + fallback is a targeted response, and the update-scope ablation (§5) tests it explicitly.
- **Length generalization literature** (positional encoding / recurrence engineering): addresses long-input inference under fixed representations; this work's fast path removes construction-length dependence at the channel level rather than the representation level.

7 Limitations

1. **Compilable set.** Only operators expressible as compositions of already-supported operations can be compiled (S1 precondition). New semantics are out of scope: the macro adds speed, not meaning.
2. **Slow-path correctness.** Compilation quality inherits the construction channel's correctness; a reference-evaluator bug propagates into supervision. Verification mitigates but does not create correctness.
3. **Verification coverage.** Equivalence is checked on held-out samples, not proven; a full guarantee would require formal verification of the compiled weights (future, Lean-based).
4. **Update-range tradeoff.** Wide updates risk disturbing learned abilities; narrow updates may limit the fast path's expressiveness. The empirical frontier is not yet mapped (ablation pending).

8 Conclusion

A model that already executes a composite operator along its construction path has everything needed to execute it fast: a small number of samples compiles the construction channel into a semantically equivalent reasoning fast path within the same model – verified, with fallback, and without expanding the semantic closure. On a 2-layer <1 MB transformer: a 2000-digit carry chain is correct on all channels, the fast path is 7× cheaper at width 2000, step-skipping generalizes with zero decay, and the macro's machine form is visible as 82→5 tokens. Neural macro compilation is the weight-level analogue of source-level macros: the semantics are the model's own; only the speed is added.

Appendix

- A. Full experiment tables: docs/paper_data/results.csv.
- B. Path-recognition examples: definition-chain expansions for addition/root/tetration.
- C. Cost protocol: construction (token_iterator), formal (engine), intuition (model forward); wall-clock per 1000 samples.
- D. Patent M1 independent-claim skeleton (semantic-path compilation).

References

[In preparation; verified full records pending. Core entries:]

- Hinton, G., Vinyals, O., Dean, J. 2015. Distilling the Knowledge in a Neural Network. arXiv:1503.02531.
- ICML 2024. Learning to Compile Programs to Neural Networks (full citation pending verification).
- P1 companion paper: Generalization as Formalized Induction (2026).
- Diffusion shortcut distillation (2025; citation pending).
- Barendregt, H. 1984. The Lambda Calculus (for definition-chain and λ foundations).